LINTON LABS · ROLLOUT REVIEW · BILLING PLATFORM FIRST

# Codex Enterprise Rollout Design

Recommendation: establish a controlled billing "golden path," prove it for two delivery cycles, then replicate the controls—not the risk—across product teams.

**Decision 1 — Start narrow, scale by evidence.** Billing is the reference environment: human-reviewed PRs only, protected branches, mandatory worktrees for concurrent agents, and release ownership remains with the service team.

## 1 · Surfaces & Windows posture

- **Default:** Codex app for parallel, repo-level tasks; it is the low-friction home for worktrees, review, and task handoff.
- **IDE extension:** default for small, developer-led edits and debugging; keep the engineer in the existing review loop.
- **CLI:** CI fixes, scripted/terminal-heavy work, and repeatable runbooks—not ad hoc production changes.
- **Cloud:** required for isolated long-running analysis, dependency upgrades, and GitHub review; forbidden for production access, customer data, or repos lacking a reviewed environment.
- **Worktrees:** mandatory for billing and any parallel change; optional for read-only triage or a single small edit.
- **Windows guardrail:** standardize Windows 11 + native `elevated` sandbox. Use `unelevated` only as a time-bounded fallback while IT fixes policy; use WSL2 semantics for Linux-native tooling or established WSL repos.

## 2 · Model & verification policy

- **Default:** Codex's standard interactive model at medium effort for routine delivery; raise to high for cross-module changes, risky billing logic, unclear failures, or security review.
- **API-key workflows:** use current Codex-tuned `gpt-5.3-codex` in the Responses API; medium is the balanced baseline, high/xhigh are exception paths.
- **Spark:** allow `GPT-5.3-Codex-Spark` only for ultra-low-latency local iteration (narrow edits, rapid prototypes, feedback loops). Do not use it alone for billing design, migrations, security-sensitive work, or autonomous cloud tasks.
- **Spark guardrail:** every Spark change must name files in scope, run the repo's targeted test/lint command, and receive a human diff review before commit.

## 3 · Context continuity & recovery

- **Checkpoint:** write a short decision log after each completed subtask, before destructive/risky operations, after 30–45 minutes, and before switching surface or owner.
- **Compaction:** compact only after recording objective, constraints, files changed, commands/results, open risks, and next command in the thread/PR.
- RECOVER Failed attempt: preserve the failed command/output, reset only the isolated worktree, state the hypothesis change, and retry once with a narrower scope.
- RECOVER Drift or lost context: stop edits; reopen from the checkpoint, compare diff to objective, discard the worktree if intent cannot be reconstructed, then restart a focused thread.

## 4 · Governance bundle: repo as policy

- **Configuration:** managed baseline in user `~/.codex/config.toml`; checked-in repo overrides in `.codex/config.toml` only for trusted projects. Pin sandbox/approval posture centrally.
- **AGENTS.md:** root file names setup + verification commands, protected paths, PR size guidance, reviewers, and "done" = tests pass + diff reviewed + no fixture/secrets churn. Add billing-specific nested instructions.
- **.rules:** allow read/search/test/lint/git-status; require approval for network, package install, writes outside worktree, force-push, deploy, and destructive commands. Block secret discovery/exfiltration and direct production tooling.
- **Skills:** ship versioned *secure PR review*, *CI fix loop*, and *release notes* skills; owners maintain examples and verification criteria.

## 5 · Cloud environment policy

- **Golden environment:** deterministic setup script installs pinned toolchains; optional maintenance script refreshes only lockfile-driven dependencies. Use shared cache deliberately; invalidate after setup, secret, or runtime changes.
- **Secrets guardrail:** secrets only in cloud secret storage and setup phase; never prompt, commit, echo, or pass secrets to agent runtime. Use ordinary environment variables only for non-sensitive configuration.
- **Internet guardrail:** agent internet off by default. Per-environment allowlist only the package registry, source host, and documented APIs required; review network-enabled task output.
- **Billing boundary:** synthetic fixtures only; no production credentials, customer exports, production database connectivity, or release execution in cloud.

## 6 · Integrations, admin & operations

- **GitHub:** begin with manual `@codex review` on billing PRs; enable automatic review only after 2 cycles of measured precision and an owner-approved ruleset. Codex advises; CODEOWNERS approve.
- **Slack / Linear:** delegate triage, summarization, issue-to-plan drafts, and read-only status. Do not delegate incident command, customer-data investigation, priority changes, approval, or release decisions. Slack task completion posts link-only by default.
- **RBAC:** separate Pilot Contributors, Billing Maintainers, Cloud Operators, Reviewers, and Workspace Admins. Cloud/environment edits and integration installation are admin-owned.
- **Signals:** adoption and cycle-time analytics; compliance logs for task, repo/environment, approvals, commands, sandbox/internet mode, diffs, and reviewer outcome. Review weekly in pilot, monthly after scale.
- **Support runbook:** check entitlement/sign-in → repo/environment selection → approval queue → sandbox mode/logs → network allowlist → missing GitHub/Slack access. Windows: verify Windows 11, admin prerequisites, active sandbox, then WSL2 choice; escalate blocked elevated setup to IT.

**Evidence base (official OpenAI documentation, accessed 18 Sep 2026):** Model guidance · Cloud environments · Windows sandbox · Configuration reference · Slack integration.

**Rollout gate:** 2 billing cycles · 0 uncontrolled production actions · measured review quality